Programação com o R

Análise de Dados Financeiros e Econômicos com o R (Perlin 2021, cap 08)

Marcelo S. Perlin

EA-UFRGS

24/09/2026

Sumário

- Introdução a Programação com o R
- Funções no R
- Loops
- Execuções Condicionais (`if`, `else`, `switch`)
- Programação Funcional com o R
- Manipulação de Dados com `dplyr` (Wickham et al. 2026)
- Tarefa final de aula

Objetivos de Aprendizagem

Objetivos de Aprendizagem

Ao final desta aula, você será capaz de:

- Criar e usar **funções** para encapsular procedimentos
- Repetir tarefas com ***loops*** e executar código condicionalmente (`if`/`else`)
- Aplicar **programação funcional** com o pacote `purrr`
- Realizar operações de agrupamento (*split-apply-combine*) com o `dplyr`

Introdução a Programação com o R

Introdução

> Programação é a execução de um conjunto de instruções complexas ao computador, as quais solucionam algum problema de análise de dados

- permite iterações sequenciais, encapsulamento de métodos, e automatização de qualquer procedimento computacional
- Em resumo, o computador será uma poderosa e escalável extensão do seu cérebro

Funções no R

Introdução

> *Everything that exists is an object. Everything that happens is a function call* — John Chambers, creator of the S language, the predecessor to R.

- Uma função representa o encapsulamento de método
- O uso de funções organiza o código, aumenta a aplicabilidade de procedimentos genéricos, e permite a rápida solução de problemas.
- um conjunto de funções é o maior ativo do pesquisador
- menos tempo reescrevendo códigos, e mais tempo pensando

Estrutura de uma função no R

- Uma função sempre possui três partes conexas:
  - entradas: informações necessárias para o processamento
  - processamento: manipulação das informações
  - saída: informações de interesse

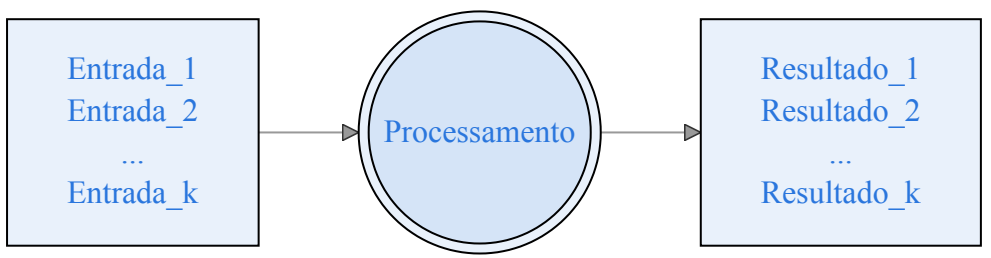
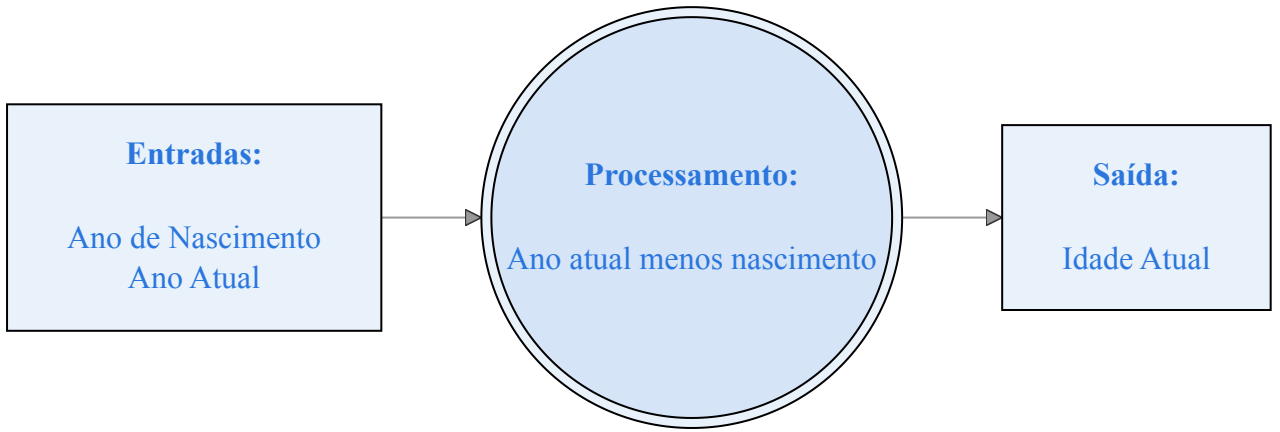


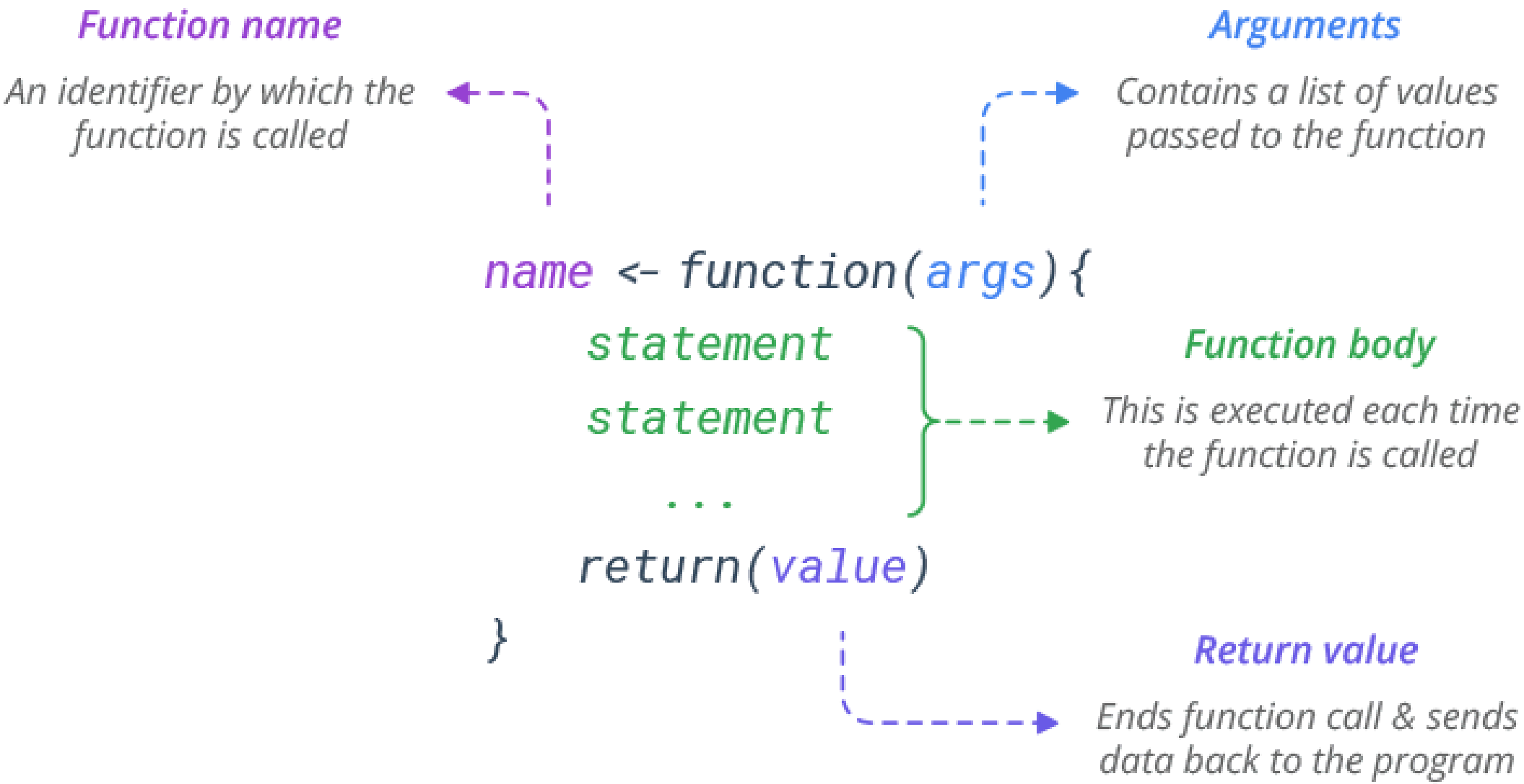
Diagrama de uma função

Qual é a sua idade?



Exemplo de diagrama de uma função

Sintaxe de uma função



Criando Funções

```r
my_fct <- function(arg1 = 1, arg2 = 'abc'){

  length_arg1 <- length(arg1)
  length_arg2 <- length(arg2)

  msg1 <- paste0('Number of elements in arg1: ', lengt
  message(msg1)

  msg2 <- paste0('Number of elements in arg2: ', lengt
  message(msg2)

  out <- c(length_arg1, length_arg2)
  return(out)
```

```
[1] 1 1
```

```r
out2 <- my_fct(arg1 = 1:10,
               arg2 = c('dab', 'asc'))
```

```
3 print(out2)
```

```
[1] 10  2
```

Um exemplo prático

> Vamos criar uma função que calcule, para qualquer ação do Yahoo Finance (`yfR`), o retorno acumulado entre 2015 e a data atual

```
get_cum_ret <- function(ticker){

  first_date <- "2015-01-01"
  last_date <- Sys.Date()

  df_yf <- yfR::yf_get(ticker, first_date, last_date)

  first_price <- dplyr::first(df_yf$price_adjusted)
  last_price <- dplyr::last(df_yf$price_adjusted)
  cum_ret <- last_price/first_price - 1

  return(cum_ret)

```

Testando a função:

```
ticker <- "PETR4.SA"
print(get_cum_ret(ticker))
```

```
[1] 20.15334
```

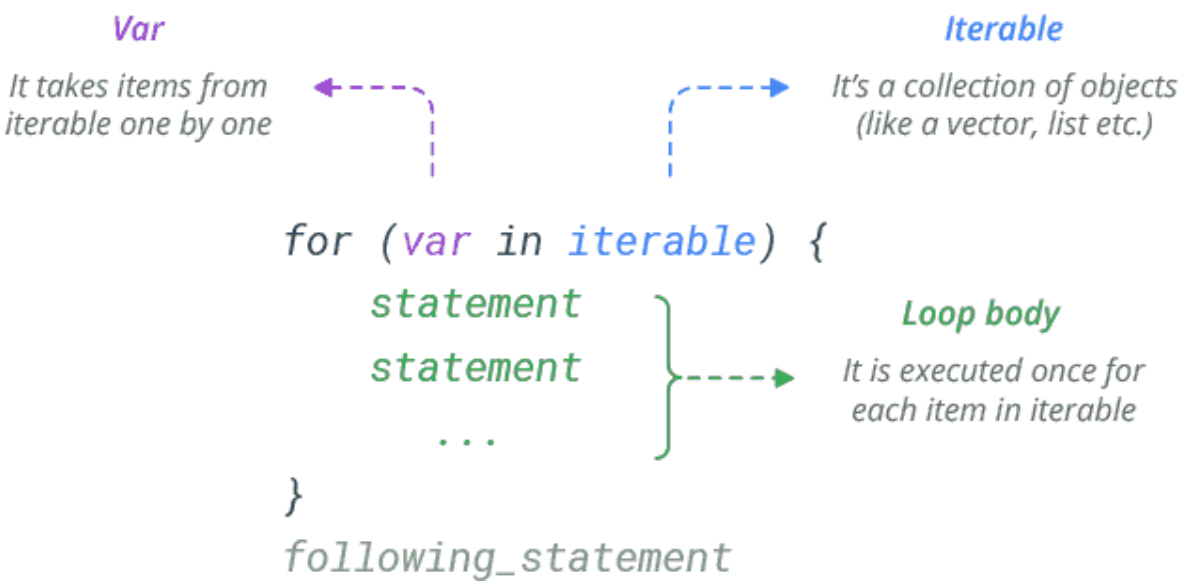
Loops

Utilizando Loops (comando *for*)

> Utilizados de forma ostensiva na análise de dados, os *loops* permitem a repetição estruturada de códigos e possibilitam o processamento de itens individualmente de uma forma fácil.

- o tamanho do *loop* é definido dinamicamente, com base em dados

A estrutura de um *loop* segue o seguinte esqueleto de código:



Um exemplo prático

```r
# set seq
my_seq <- seq(-5,5)

# do loop
for (i in my_seq){
  message(paste('The value of i is',i))
}
```

O uso de *loops* encadeados, isto é, um *loop* dentro de outro *loop*, também é possível. Veja o exemplo a seguir, onde apresentamos todos os elementos de uma matriz:

```r
# set matrix
my_mat <- matrix(1:9, nrow = 3)

# loop all values of matrix
for (i in seq(1, nrow(my_mat))){
  for (j in seq(1, ncol(my_mat))){
```

```
7       message(paste0('Element [', i, ', ', j, '] = ', my
8     }
9   }
```

Coletando retornos acumulados com loops

```
ticker_vec <- c("PETR4.SA", "GRND3.SA", "WEGE3.SA", "A

cum_ret_vec <- numeric()
for (i_ticker in seq_along(ticker_vec) ) {

  this_ticker <- ticker_vec[i_ticker]
  this_cum_ret <- get_cum_ret(this_ticker)

  cum_ret_vec[this_ticker] <- this_cum_ret
}

cum_ret_vec
```

```
  PETR4.SA   GRND3.SA   WEGE3.SA   AMER3.SA       ^BVSP
20.1533378  1.0704272  9.8883402 -0.9974928  2.8302688
```

Usando dados em loops

```r
sp_comp <- yfR::yf_index_composition("SP500")

ticker_vec <- sp_comp$ticker[1:5]

cum_ret_vec <- numeric()
for (i_ticker in seq_along(ticker_vec) ) {

  this_ticker <- ticker_vec[i_ticker]
  this_cum_ret <- get_cum_ret(this_ticker)

  cum_ret_vec[this_ticker] <- this_cum_ret
}

```

```
      MMM       AOS       ABT      ABBV       ACN
0.7969428 1.5011226 1.8944848 5.4466193 1.5224930
```

Exemplo de loop e agrupamento de dados

- Outro exemplo prático do uso de um *loop* é o processamento de dados de acordo com grupos. Por exemplo, para dados de preços de diferentes ações do índice SP500, caso quiséssemos calcular o preço médio de cada ação, construiríamos o seguinte código para importar os dados e processá-los:

```
1  library(dplyr)
2
3  # read data
4  my_f <- afedR3::data_path('CH08_some-stocks-SP500.csv'
5  my_df <- readr::read_csv(my_f)
6
7  # find unique tickers in column ticker
8  unique_tickers <- unique(my_df$ticker)
9
10 # create empty df
11 tab_out <- tibble()
12
```

```
13  # loop tickers
```

```
# A tibble: 10 × 3
   ticker mean_price last_date
   <chr>        <dbl> <date>
 1 BA           256.  2022-12-30
 2 CMI          180.  2022-12-30
 3 DE           238.  2022-12-30
 4 FCX           22.2 2022-12-30
 5 K             59.4 2022-12-30
 6 MSI          170.  2022-12-30
 7 NTRS          92.3 2022-12-30
 8 PNR           47.2 2022-12-30
```

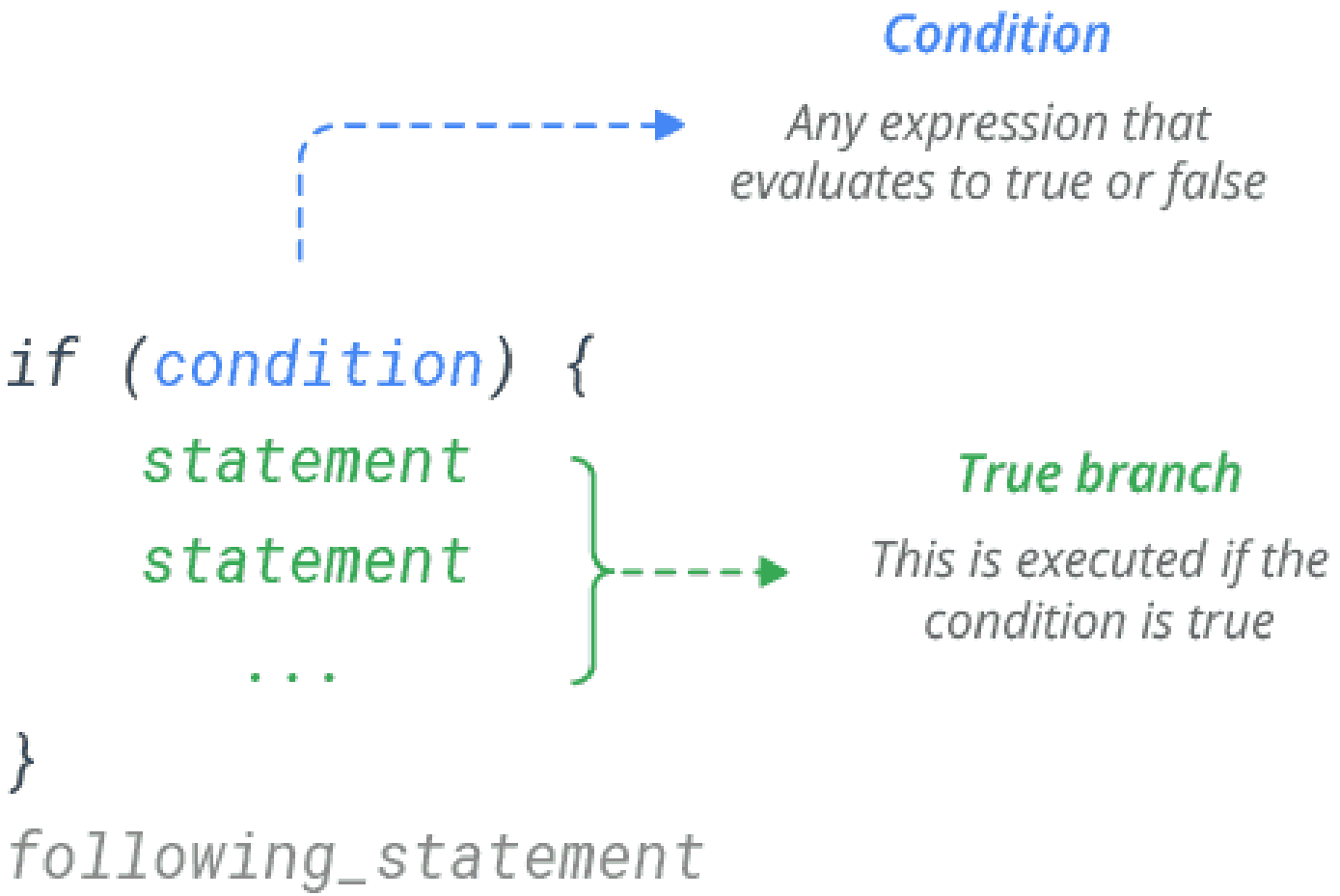
Dica de desempenho

Crescer `tab_out` com `bind_rows` a cada iteração funciona, mas é ineficiente. Em produção, colete os resultados em uma lista e combine-os uma única vez ao final do *loop* (ou use `purrr::map()` combinado com `dplyr::bind_rows()`).

Execuções Condicionais (`if`, `else`, `switch`)

Introdução

> Execuções condicionais permitem tomar decisões binárias no código, do tipo **se, então** (*if, then*)



Um exemplo com loop

```
my_x <- 1:10
my_thresh <- 5

for (i in my_x){
  if (i > my_thresh){
    message('Value of i is ', i, ' - Higher than ', my
  } else {
    message('Value of i is ', i, ' - Lower or equal th
  }
}
```

Podemos, também, aplicar mais de uma condição lógica com o comando `else` e `else if`. Veja a seguir:

```
my_x <- 1:10
my_thresh <- 5

```

```
4  for (i in my_x){
5    if (i > my_thresh){
6      message('Value of i is ', i, ' - Higher than ', my
7    } else if (i == my_thresh) {
8      message('Value of i ', i, ' - equal to ', my_thres
9    } else {
10     message('Value of i ', i, ' - lower than ', my_thr
11   }
12 }
```

Programação Funcional com o R

Introdução

> Programação funcional é o uso exclusivo de funções para resolver algum problema de análise de dados.

- Pontos positivos:
  - código fica mais limpo e organizado
  - permite controle automático de erros de execução
  - código mais fácil de manter e de ser reutilizado
- Pontos negativos:
  - maior curva de aprendizado para iniciantes
  - erros de execução podem ser mais difíceis de depurar

Comparação de código

```
1 fct <- function(x) {
2   return(x+1)
3 }
4
5 y_vec <- 1:10
```

Versão com loop

```
1 x_results <- numeric(leng
2 for (i in seq_along(y_vec
3
4   x_results[i] <- fct(y_v
5 }
6
7 x_results
```

```
 [1]  2  3  4  5  6  7  8  9
10 11
```

Versão com `purrr::map_dbl`

```
1 x_results <- purrr::map_d
2 x_results
```

```
 [1]  2  3  4  5  6  7  8  9
10 11
```

ou:

```
1 x_results <- y_vec |>
2   purrr::map_dbl(fct)
3 x_results
```

```
 [1]  2  3  4  5  6  7  8  9
10 11
```

As funções de programação funcional

- Pacote `base` oferece funções nativas da família *apply*: `sapply`, `lapply`, `tapply`, `mapply`, `apply` e `by`.
- No `tidyverse`, programação funcional é oferecida pelo pacote `purrr` (Wickham and Henry 2026)
- O foco aqui será com as funções do pacote `purrr`

Pacote `purrr`

Principais funções:

- `map()`: retorna sempre uma lista
- `map_dbl()`: retorna sempre um vetor numérico (*double*)
- `map_int()`: retorna sempre um vetor de inteiros (*integer*)
- `map_chr()`: retorna sempre um vetor de texto (*character*)
- `map_lgl()`: retorna sempre um vetor lógico (*logical*)
- `pmap()`: retorna uma lista, mas com entrada flexível de argumentos

Função purrr::map()

```r
library(purrr)

# set list
my_l <- list(vec1 = 1:10,
             vec2 = 1:50,
             vec3 = 1:5,
             char1 = letters[1:10])

# get length of objects
res_out <- my_l |>
  map_int(length)

print(res_out)
```

```
vec1  vec2  vec3 char1
  10    50     5    10
```

```r
# find character objets
res_out <- my_l |>
  map_lgl(is.character)

print(res_out)
```

```
 vec1  vec2  vec3 char1
FALSE FALSE FALSE  TRUE
```

Outro ponto interessante nas funções do `purrr` é que permitem, de uma forma simples, o acesso a elementos de uma lista. Para isso, basta indicar uma entrada de posição ou nome na função `map`:

```r
# set list
my_l <- list(vec1 = c(elem1 = 10, elem2 = 20, elem3 =
             char1 = c(elem1 = 40, elem2 = 50, elem3 =

# get second element of each element in list, by posit
res_out <- my_l |> map(2)
print(res_out)
```

```
$vec1
[1] 20

$char1
[1] 50
```

```r
# get third element of each element in list, by name
res_out <- my_l |> map('elem3')
print(res_out)
```

```
$vec1
[1] 5

$char1
[1] 15
```

Função purrr::safely()

> permite o controle e tratamento de erros de execução

Exemplo:

- função retorna com erro de execução (não é possível somar 1 a um texto)

```
1 library(purrr)
2
3 add_one <- function(x) {
4   return(x + 1)
5 }
6
7 add_one('a')
```

```
Error in `x + 1`:
! non-numeric argument to binary operator
```

Agora vamos utilizar `safely` para encapsular `add_one` em outra função chamada `add_one_safely`:

```
1 # with safely
2 add_one_safely <- safely(add_one)
3
4 print(add_one_safely('a'))
```

```
$result
NULL

$error
<simpleError in x + 1: non-numeric argument to binary
operator>
```

Um exemplo de uso completo:

```
1 my_l <- list(1:5,
2               'a',
3               1:4)
4
```

```r
res_out <- my_l |>
  map(safely(add_one))

print(res_out)
```

```
[[1]]
[[1]]$result
[1] 2 3 4 5 6

[[1]]$error
NULL



[[2]]
[[2]]$result
NULL
```

Podemos facilmente ver que a função teve erro no segundo elemento de `my_l`. Indo além, caso quiséssemos apenas os resultados, podemos escrever:

```r
print(res_out |> map('result'))
```

```
[[1]]
[1] 2 3 4 5 6

[[2]]
NULL

[[3]]
[1] 2 3 4 5
```

Ou apenas as mensagens de erro:

```
1 print(res_out |> map('error'))
```

```
[[1]]
NULL

[[2]]
<simpleError in x + 1: non-numeric argument to binary
operator>
```

```
[[3]]
NULL
```

A saída padrão para o caso de erros também pode ser modificada. Veja o próximo caso, onde inserimos o valor `NA` toda vez que a função resultar em erro (argumento `otherwise`), e retornamos um vetor:

```
my_l <- list(1,
             'a',
             4)

# NA for errors
res_out <- my_l |>
  map(safely(add_one,
             otherwise = NA)) |>
  map_dbl('result')

# print result
print(res_out)
```

```
[1]  2 NA  5
```

Função purrr::pmap()

- funções `purrr::map_*` permitem a passagem de apenas um argumento dinâmico (que muda) em cada chamada da função
- função `pmap` permite passar qualquer número de argumentos para uma função de destino, inclusive replica loops aninhados.

Como exemplo, vamos considerar uma função que cria uma frase a partir de três entradas:

```
1 build_phrase <- function(name_in, fruit_in, verb_in) {
2   my_msg <- paste0('My name is ', name_in,
3                    ' and I like to eat ', fruit_in,
4                    ' while ', verb_in, '.')
5
6   return(my_msg)
7 }
```

```
8
9 build_phrase('Joe', 'apple', 'studying')
```

```
[1] "My name is Joe and I like to eat apple while
studying."
```

Usando loops aninhados, escrevemos o seguinte código:

```
1  names_vec <- c('Joe', 'Kate')
2  fruits_vec <- c('kiwi', 'apple')
3  verb_vec <- c('rowing', 'studying')
4
5  my_phrases <- character()
6  for (i_name in names_vec) {
7    for (i_fruit in fruits_vec) {
8      for (i_verb in verb_vec) {
9        my_phrases <- c(my_phrases,
10                       build_phrase(i_name, i_fruit, i_
11     }
12   }
13 }
```

```
[1] "My name is Joe and I like to eat kiwi while rowing."
[2] "My name is Joe and I like to eat kiwi while
studying."
[3] "My name is Joe and I like to eat apple while
rowing."
[4] "My name is Joe and I like to eat apple while
studying."
[5] "My name is Kate and I like to eat kiwi while
rowing."
[6] "My name is Kate and I like to eat kiwi while
studying."
```

Embora o código funcione conforme o esperado, uma melhor abordagem é usar `purrr::pmap`. Tudo o que precisamos fazer é passar todas as combinações de argumentos para a função:

```
df_grid <- tidyr::expand_grid(names_vec = names_vec,
                              fruits_vec = fruits_vec,
                              verb_vec = verb_vec)

l_args <- list(name_in = df_grid$names_vec,
```

```
                  fruit_in = df_grid$fruits_vec,
                  verb_in = df_grid$verb_vec)

my_phrases <- purrr::pmap_chr(.l = l_args,
                              .f = build_phrase)

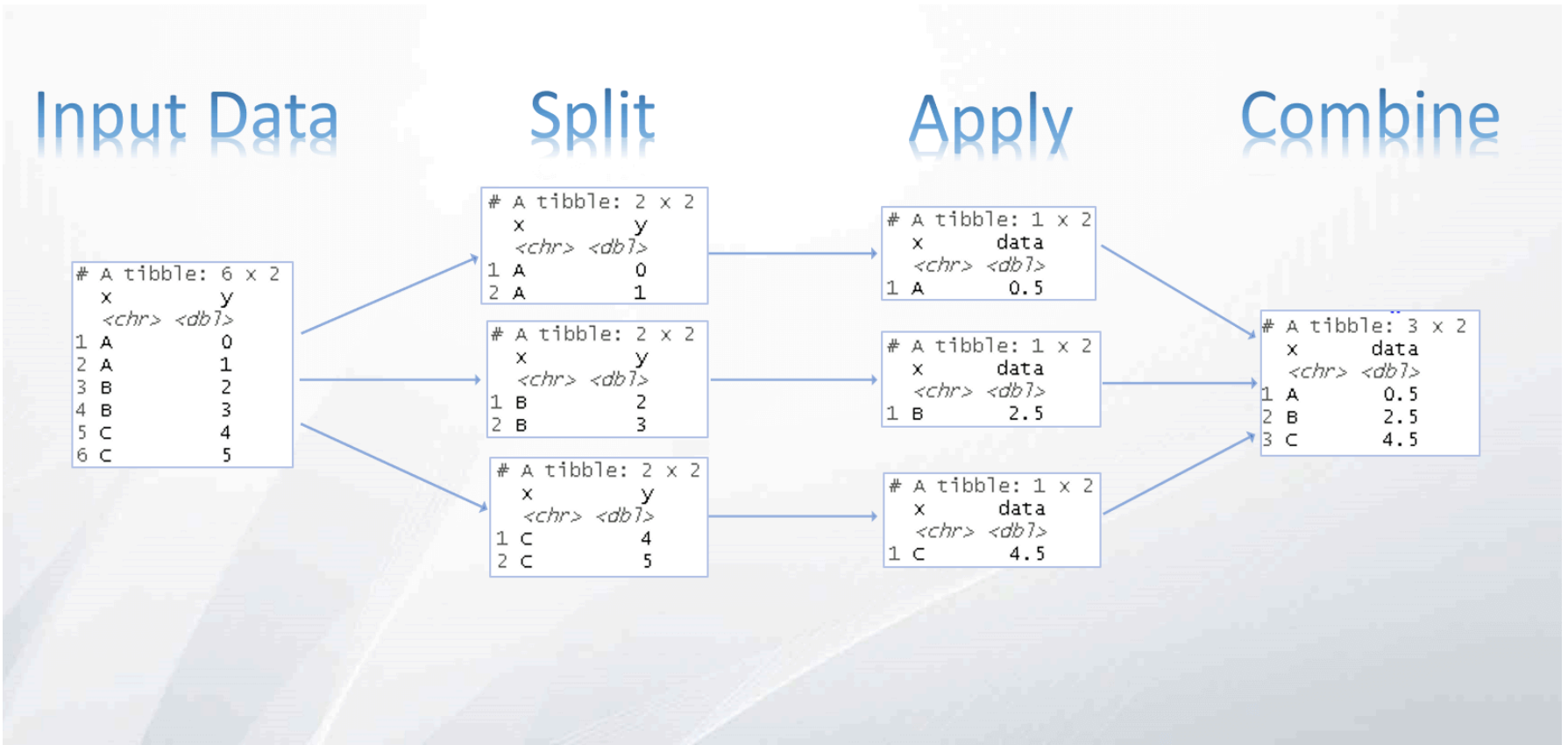
print(my_phrases)
```

```
[1] "My name is Joe and I like to eat kiwi while rowing."
[2] "My name is Joe and I like to eat kiwi while
studying."
[3] "My name is Joe and I like to eat apple while
rowing."
[4] "My name is Joe and I like to eat apple while
studying."
[5] "My name is Kate and I like to eat kiwi while
rowing."
[6] "My name is Kate and I like to eat kiwi while
studying."
```

Manipulação de Dados com `dplyr` (Wickham et al. 2026)

Introdução

- uma das maiores contribuições do `tidyverse`
- pacote `dplyr` permite a realização de análises complexas do tipo *split-apply-combine* com uma sintaxe simples e intuitiva.



Operações de Grupo com `dplyr`

Etapas:

1. agrupar de acordo com valores de uma coluna
2. realizar algum cálculo nos dados filtrados
3. agregar os dados para formar uma nova tabela

Exemplo

> vamos calcular a média, desvio padrão, máximo e mínimo dos retornos de um grupo de ações

```
 1 library(dplyr)
 2
 3 tickers <- c("PETR4.SA", "VALE3.SA", "GRND3.SA")
 4 first_date <- "2018-01-01"
 5
 6 tbl_yf <- yfR::yf_get(tickers, first_date) |>
 7   na.omit() |>
 8   group_by(ticker) |>
 9   summarise(ret_mean = mean(ret_adjusted_prices),
10             ret_sd = sd(ret_adjusted_prices),
11             ret_max = max(ret_adjusted_prices),
12             ret_min = min(ret_adjusted_prices))
13
```

```
# A tibble: 3 × 5
  ticker    ret_mean ret_sd ret_max ret_min
  <chr>        <dbl>  <dbl>   <dbl>   <dbl>
1 GRND3.SA 0.000181 0.0204  0.0990  -0.112
2 PETR4.SA 0.00149  0.0258  0.222   -0.297
3 VALE3.SA 0.000809 0.0221  0.214   -0.246
```

Um exemplo mais complexo

```r
tbl_yf <- yfR::yf_get(tickers, first_date) |>
  na.omit() |>
  mutate(weekday = weekdays(ref_date)) |>
  group_by(ticker, weekday) |>
  summarise(ret_mean = mean(ret_adjusted_prices),
            ret_sd = sd(ret_adjusted_prices),
            ret_max = max(ret_adjusted_prices),
            ret_min = min(ret_adjusted_prices))


tbl_yf
```

```
# A tibble: 15 × 6
# Groups:   ticker [3]
   ticker   weekday    ret_mean ret_sd ret_max ret_min
   <chr>    <chr>         <dbl>  <dbl>   <dbl>   <dbl>
 1 GRND3.SA Friday    -0.000466 0.0230  0.0811 -0.111
```

2	GRND3.SA	Monday	-0.000981	0.0204	0.0990	-0.112
3	GRND3.SA	Thursday	-0.000210	0.0191	0.0788	-0.0792
4	GRND3.SA	Tuesday	0.00160	0.0188	0.0775	-0.0542
5	GRND3.SA	Wednesday	0.000953	0.0203	0.0871	-0.0782
6	PETR4.SA	Friday	-0.000988	0.0261	0.222	-0.149

Resumo

Resumo

- **Funções** encapsulam entradas → processamento → saída e são o maior ativo do pesquisador
- **Loops** (`for`) repetem instruções de forma dinâmica e são úteis para processar grupos de dados
- **Condicionais** (`if`/`else`) permitem tomar decisões no código
- **Programação funcional** (`purrr`) substitui loops por `map_*`/`pmap_*`
- **`dplyr`** resolve *split-apply-combine* com `group_by()` + `summarise()`

Tarefa final de aula

Tarefa 04

1. Crie uma função chamada `say_my_name()` que tome como entrada um nome de pessoa e mostre na tela o texto *Your name is* .... Dentro do escopo da função, utilize comentários para descrever o propósito da função, suas entradas e saídas.
2. Implemente um teste para os objetos de entrada, de forma que, quando o nome de entrada não for da classe character, um erro é retornado ao usuário. Teste sua nova função e verifique se a mesma está funcionando conforme esperado.
3. Crie um vetor com cinco nomes quaisquer, chamado *my_names*. Utilizando um loop, aplique função say_my_name para cada elemento de my_names.
4. No banco de dados do Brasil.IO[1] encontrarás uma tabela com nomes e gêneros derivados de uma das pesquisas do IBGE. Importe os dados do arquivo para R e, usando um *loop*, aplique a função `say_my_name` a 15 nomes aleatórios do banco de dados. Dica: neste caso, você pode baixar os dados direto do link usando `readr::read_csv(LINK)`.
5. Use o pacote `yfR` para baixar dados do índice SP500 (`'^GSPC'`), Ibovespa (`'^BVSP'`), FTSE (`'^FTSE'`) e Nikkei 225 (`'^N225'`) de `'2010-01-01'` até a data atual. Com os dados importados, use um *loop* para calcular o retorno médio, máximo e mínimo de cada índice durante o período analisado. Salve todos os resultados em uma tabela única e a mostre no *prompt* do R.

Referências

Perlin, Marcelo S. 2021. *adfeR: Data, Exercises and Functions for Book "Analise de Dados Financeiros e Economicos Com o r"*. https://github.com/msperlin/adfedR/.

Wickham, Hadley, Romain François, Lionel Henry, Kirill Müller, and Davis Vaughan. 2026. *Dplyr: A Grammar of Data Manipulation*. https://dplyr.tidyverse.org.

Wickham, Hadley, and Lionel Henry. 2026. *Purrr: Functional Programming Tools*. https://purrr.tidyverse.org/.